

CONNECTIONS BETWEEN SCHEDULE-FREE OPTIMIZERS, AdEMAMix, AND ACCELERATED SGD VARIANTS

Anonymous authors
Anonymous submission

ABSTRACT

A recent family of deep-learning optimizers—Schedule-Free methods, AdEMAMix, MARS, and Lion—departs from the classical single-buffer momentum recursion by restructuring how past gradients are accumulated and reweighted in the parameter update. Independently, the theory of accelerated stochastic gradient descent (SGD) in the noise-dominated regime attains improved convergence rates by decoupling the momentum coefficient from the scalar weight applied to the current stochastic gradient within the update rule. This paper establishes explicit, update-rule-level equivalences between these two developments: we show that each of the above optimizers can be expressed as an instance of a common accelerated-SGD update, optionally composed with preconditioning and weight averaging. We substantiate the analysis empirically on a 150M-parameter language-modeling task, and observe that AdEMAMix—whose update most closely matches the accelerated-SGD schedules—attains the lowest validation loss among the compared methods. Motivated by this correspondence, we introduce Simplified-AdEMAMix, a variant that collapses the two momentum buffers of AdEMAMix into a single buffer with an explicit fixed weight on the current gradient, and demonstrate that it matches AdEMAMix in both the large- and small-batch regimes without incurring the additional momentum state. The implementation of Simplified-AdEMAMix will be released upon acceptance.

1 INTRODUCTION

A number of optimization algorithms have recently been proposed for deep learning, including Lion (Chen et al., 2023), Schedule-Free SGD/AdamW (Defazio et al., 2024), and AdEMAMix (Pagliardini et al., 2024). Although each was derived from a distinct motivation, they share a precise structural property: every one of them alters the momentum recursion—that is, the rule by which past stochastic gradients are aggregated into the parameter update—rather than the preconditioner or the learning-rate schedule alone.

In parallel, a body of theoretical work has characterized the acceleration of gradient descent under stochastic noise. Classical momentum schemes—heavy-ball and Nesterov momentum—provably accelerate *deterministic* gradient descent, in particular on quadratic objectives, but do not improve the asymptotic convergence rate of SGD (Jain et al., 2018; Liu & Belkin, 2020). This gap has motivated a family of alternative momentum recursions designed to recover acceleration in the presence of gradient noise (Jain et al., 2018; Vaswani et al., 2019; Liu & Belkin, 2020; Gupta et al., 2023). A unifying observation is that every such accelerated-SGD scheme can be cast as decoupling the momentum coefficient from the scalar weight applied to the current gradient in the update—two quantities that are tied together in standard momentum SGD.

Our principal contribution is to make the correspondence between these two lines of work explicit at the level of the update equations. Concretely, we prove that Schedule-Free SGD is algebraically equivalent to accelerated SGD composed with weight averaging, and we show that Lion, Schedule-Free AdamW, and AdEMAMix are each obtained by composing a preconditioner with an accelerated-SGD update. Although fragments of these relationships have been noted previously (Defazio, 2021), to the best of our knowledge the full update-rule-level correspondence between these recent optimizers and accelerated SGD has not been formally established.

We validate the analysis on a 150M-parameter decoder-only transformer trained on 15B tokens with a small batch size of 32k tokens, a configuration chosen to place training firmly in the noise-dominated regime where acceleration is theoretically expected to help. Consistent with the derived equivalence, the validation trajectory of Schedule-Free AdamW closely tracks that of the accelerated-SGD-based AdamW update of Algorithm 3, and the accelerated

methods yield a modest but consistent improvement over their non-accelerated counterparts at this batch size. As the batch size is increased into the low-noise regime, this advantage vanishes—precisely the behavior predicted by the correspondence to accelerated SGD.

Our main contributions are stated below:

1. We derive exact, update-rule-level equivalences between accelerated SGD and the recently proposed optimizers Schedule-Free SGD and AdEMAMix, identifying the coefficient maps under which each reduces to the general accelerated-SGD form of Equation (1).
2. We provide empirical validation on a 150M-parameter decoder-only transformer, benchmarking AdamW, Schedule-Free AdamW, AdEMAMix, and MARS under matched compute. AdEMAMix, whose update coefficients most closely instantiate the accelerated-SGD schedules, attains the lowest validation loss among these methods.
3. Consistent with the equivalence to accelerated SGD, the advantage of these methods over Adam disappears in the large-batch (low-noise) regime; moreover, we show that Adam equipped with a momentum schedule recovers the performance of AdEMAMix.
4. In the large-batch regime, Schedule-Free AdamW is substantially outperformed by AdamW with cosine decay, a gap we trace to the structural coupling between the momentum coefficient and the weight-averaging coefficient that Schedule-Free optimizers impose by construction.
5. We propose Simplified-AdEMAMix, a variant that retains a single momentum buffer with an explicit fixed weight on the current gradient, and show that it matches AdEMAMix across both the small- and large-batch regimes while removing the second momentum buffer.

2 RELATED WORK

We situate our analysis within two strands of prior work: the theory of accelerated SGD variants and the practical optimizers whose update rules our equivalences target.

On the theoretical side, Jain et al. (2018) proposed an accelerated SGD scheme with provably improved convergence rates for least-squares regression. Kidambi et al. (2018) reduced this scheme to a simpler update and established formally that heavy-ball/Nesterov momentum alone cannot accelerate SGD in this setting. Subsequent work (Liu & Belkin, 2020; Vaswani et al., 2019; Gupta et al., 2023) extended these guarantees to general convex and strongly convex objectives under a range of smoothness and noise assumptions.

On the algorithmic side, several optimizers exhibit update structures closely related to these accelerated schemes. Lucas et al. (2019) form the update from a weighted combination of several momentum buffers maintained at distinct decay coefficients. Ma & Yarats (2019) construct an optimizer directly motivated by the analysis of Jain et al. (2018), interpolating between the current gradient and a momentum term. More recently, Chen et al. (2023) obtained an optimizer via symbolic/genetic search that, like the accelerated-SGD variants, applies different scalar weights to the current gradient and to the momentum buffer in its update. Pagliardini et al. (2024) instead maintains two momentum buffers at different time scales and combines them linearly in the final update.

3 BACKGROUND

3.1 MOMENTUM

Momentum is a standard mechanism for accelerating the convergence of gradient descent on deterministic objectives. For iterate w_t and momentum coefficient β , the heavy-ball update maintains a running gradient accumulator m_t and takes a step along it:

$$m_t = \beta m_{t-1} + \nabla f(w_t); \quad w_t = w_{t-1} - \eta m_t$$

3.2 WEIGHT AVERAGING

Weight averaging is a variance-reduction technique for stochastic optimization: rather than returning the final iterate w_T , one returns a running convex combination $\bar{w}_T$ of the iterate trajectory, with per-step mixing weights γ_t . The average is maintained by the recursion:

$$\bar{w}_T = (1 - \gamma_T)\bar{w}_{T-1} + \gamma_T w_T$$

Throughout this paper, every instance of weight averaging uses mixing coefficients γ_t of the form:

$$\gamma_t \approx 1 - \frac{1}{\delta t}$$

for some constant $0 \leq \delta \leq 1$, which corresponds to averaging over a trailing window comprising a fixed fraction δ of the iterates.

3.3 ACCELERATED SGD

We now state a general parametric family that subsumes the accelerated-SGD methods considered in this work:

$$m_t = \beta_{a,t} m_{t-1} + g_t, \quad w_{t+1} = w_t - \eta_{a,t} m_t - \alpha_{a,t} g_t \quad (1)$$

Here $\beta_{a,t}, \alpha_{a,t}, \eta_{a,t}$ are scalar coefficients, permitted to depend on the iteration index t , and g_t denotes the stochastic gradient evaluated at w_t . The subscript ‘a’ marks coefficients belonging to this accelerated-SGD parameterization. The defining feature of the family is that the current gradient g_t enters the parameter update twice—once through the momentum buffer m_t with weight $\eta_{a,t}$, and once directly with an independent weight $\alpha_{a,t}$ —so that the momentum coefficient $\beta_{a,t}$ is decoupled from the effective weight on the current gradient.

Two observations follow immediately. First, imposing $\alpha_{a,t} = 0$ collapses the family to standard momentum SGD, in which the two weights are no longer independent. Second, the accelerated-SGD algorithms of Jain et al. (2018); Vaswani et al. (2019); Liu & Belkin (2020); Gupta et al. (2023) are each recovered by an appropriate choice of the coefficient triple $(\beta_{a,t}, \eta_{a,t}, \alpha_{a,t})$; the explicit coefficient maps establishing this equivalence are derived in Appendix B.

4 CONNECTIONS BETWEEN EXISTING OPTIMIZERS AND ACCELERATED SGD

In this section we derive, at the level of the update equations, the exact correspondence between existing optimizers—Schedule-Free methods and AdEMAMix—and the accelerated-SGD family of Equation (1). These derivations expose which coefficient of each optimizer plays the role of the decoupled gradient weight, and they motivate a simplified variant of AdEMAMix that retains a single momentum buffer while matching the performance of AdEMAMix in both the small- and large-batch regimes.

4.1 SCHEDULE-FREE SGD

Schedule-Free SGD (Defazio et al., 2024) is a constant-learning-rate optimizer designed to remove the need for an explicit learning-rate schedule. Adopting the notation of Defazio et al. (2024), its state consists of three coupled sequences governed by the recursions:

$$\begin{aligned} y_t &= (1 - \beta)z_t + \beta x_t \\ z_{t+1} &= z_t - \gamma g(y_t) \\ x_{t+1} &= (1 - c_{t+1})x_t + c_{t+1}z_{t+1} \end{aligned}$$

Here y_t is the point at which the stochastic gradient is evaluated, and x_t is the averaged iterate returned for evaluation. To expose the accelerated-SGD structure, we reparameterize the dynamics in terms of y_t and an auxiliary momentum variable m_t , defined by:

$$m_{t+1} = \frac{x_t - z_{t+1}}{\gamma}. \quad (2)$$

Substituting the recursion for z_{t+1} and telescoping, m_t simplifies to a first-order momentum recursion:

$$m_t = \frac{x_t - z_{t+1}}{\gamma} \quad (3)$$

$$= \frac{x_t + \gamma g_t - z_t}{\gamma} \quad (4)$$

$$= \frac{x_t - z_t + \gamma g_t}{\gamma} \quad (5)$$

$$= \frac{(1 - c_t)(x_{t-1} - z_t) + \gamma g_t}{\gamma} \quad (6)$$

$$= (1 - c_t)m_{t-1} + g_t. \quad (7)$$

Hence m_t obeys the momentum recursion of Equation (1) under the identification $\beta_{a,t} = 1 - c_t$. With m_t so defined, we next derive the update induced on y_t :

$$y_{t+1} = (1 - \beta)z_{t+1} + \beta x_{t+1} \quad (8)$$

$$= (1 - \beta)(z_t - \gamma g_t) + \beta((1 - c_{t+1})x_t + c_{t+1}z_{t+1}) \quad (9)$$

$$= (1 - \beta)z_t + \beta x_t - (1 - \beta)\gamma g_t + \beta c_{t+1}(z_{t+1} - x_t) \quad (10)$$

$$= y_t - \gamma[\beta c_{t+1}m_t + (1 - \beta)g_t]. \quad (11)$$

Therefore y_t satisfies the parameter update of Equation (1) with $w_t = y_t$, $\eta_{a,t} = \gamma\beta c_{t+1}$, and $\alpha_{a,t} = \gamma(1 - \beta)$; that is, the gradient-evaluation sequence of Schedule-Free SGD is exactly an accelerated-SGD iterate. It remains to characterize the evaluation sequence x_t , which we analyze next:

$$x_{t+1} = (1 - c_{t+1})x_t + c_{t+1}z_{t+1}$$

$$x_{t+1} = (1 - c_{t+1})x_t + c_2 \left(\frac{y_{t+1} - \beta x_{t+1}}{1 - \beta} \right)$$

$$x_{t+1}(1 - \beta + c_{t+1}\beta) = (1 - c_{t+1})(1 - \beta)x_t + c_{t+1}y_{t+1}$$

$$x_{t+1} = \frac{(1 - c_{t+1})(1 - \beta)x_t + c_{t+1}y_{t+1}}{(1 - c_{t+1})(1 - \beta) + c_{t+1}}.$$

Thus x_t is a convex combination of the accelerated iterates y_t ; expanding the recursion shows that for constant c_t it reduces to an exponential moving average of y_t . Together, these two derivations establish the following equivalence: Schedule-Free SGD is exactly accelerated SGD (on y_t) composed with weight averaging (yielding x_t).

This decomposition attributes the efficacy of Schedule-Free SGD to two separable mechanisms:

1. acceleration relative to standard momentum SGD, inherited from the equivalence of the y_t -sequence to accelerated SGD;
2. the admissibility of a constant learning rate without a schedule, supplied by the (trailing) weight averaging that produces x_t ; see Section 4.1.3.

Two implementation-level advantages are specific to Schedule-Free SGD/Adam:

- weight averaging incurs no additional optimizer state, since x_t is maintained implicitly by the recursion rather than as a separate buffer;

- the weight-averaging coefficient is not exposed as a free hyperparameter, being determined by β and c_t .

This economy comes at a cost: because a single set of coefficients simultaneously governs momentum and averaging, the two cannot be tuned independently. In Section 5.1 we show that this structural coupling degrades performance in the large-batch regime.

4.1.1 LIMITING CASE $\beta = 0.0$

Setting $\beta = 0$ suppresses the accelerated component: as noted in (Defazio et al., 2024), the recursion collapses to plain SGD followed by weight averaging. With $c_t = 1/t$, the average incorporates every iterate from initialization, i.e., uniform averaging over the full trajectory.

4.1.2 LIMITING CASE $\beta = 1.0$

Setting $\beta = 1$ suppresses the averaging component: as noted in (Defazio et al., 2024), the recursion collapses to standard momentum SGD, with the effective momentum coefficient $\beta_{a,t}$ scaling as $1 - 1/t$.

4.1.3 DEFAULT CONFIGURATION $\beta = 0.9$

The default setting $\beta = 0.9$ interpolates between these extremes, and yields the following regime:

- with c_t scaling as $1/t$, the momentum coefficient grows as $1 - 1/t$;
- the ratio of the direct gradient weight to the momentum weight is held fixed at $(1 - \beta)/(\beta c_{t+1}) \approx 0.11$, independent of t ;
- the effective averaging window spans approximately the most recent 10% of the iterates.

4.2 LION

Lion (Chen et al., 2023) maintains two exponential moving averages of the gradient at coefficients β_1 and β_2 , applying the coordinate-wise sign of one to form the step while updating the other as the persistent momentum state:

$$m'_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$\theta_t = \theta_{t-1} - \eta \text{sign}(m'_t)$$

$$m_t = \beta_2 m_{t-1} + (1 - \beta_2) g_t.$$

Because the step direction m'_t mixes the current gradient and the momentum buffer with the two independent weights $1 - \beta_1$ and β_1 , the pre-sign update of Lion (Chen et al., 2023) is an instance of the accelerated-SGD family of Equation (1); the full update is that instance composed with the coordinate-wise sign nonlinearity.

4.3 MARS

We now show that the practical variant of MARS (Yuan et al., 2024), denoted MARS-Approx, is an instance of the accelerated-SGD family composed with an Adam-style preconditioner. Omitting bias correction and gradient clipping, its update reads:

$$\begin{aligned} c_t &= g_t + \gamma \frac{\beta_1}{1 - \beta_1} [g_t - g_{t-1}] \\ m_t &= \beta_1 m_{t-1} + (1 - \beta_1) c_t \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) c_t^2 \\ x_{t+1} &= x_t - \eta \frac{m_t}{\sqrt{v_t} + \epsilon} \end{aligned}$$

where m_t is the first-moment (momentum) estimate, v_t is the second-moment estimate used for preconditioning, and x_t is the parameter vector. Introducing the shifted momentum $\hat{m}_t = m_t - \gamma g_t$, the update can be rewritten as:

$$\begin{aligned} c_t &= g_t + \gamma \frac{\beta_1}{1 - \beta_1} [g_t - g_{t-1}] \\ \hat{m}_t &= \beta_1 \hat{m}_{t-1} + (1 - \beta_1)(1 - \gamma)g_t \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2)c_t^2 \\ x_{t+1} &= x_t - \eta \frac{\hat{m}_t + \gamma g_t}{\sqrt{v_t} + \epsilon} \end{aligned}$$

In this form the numerator $\hat{m}_t + \gamma g_t$ exhibits the accelerated-SGD structure of Equation (1): the momentum buffer $\hat{m}_t$ and the current gradient g_t enter with independent weights. The distinction from an accelerated AdamW lies in the preconditioner—MARS forms v_t from the correction term c_t rather than from g_t —and we quantify the empirical consequence of this choice in Section 5.

4.4 AdEMAMix

AdEMAMix (Pagliardini et al., 2024) is structurally close to an accelerated-SGD-based AdamW, but realizes the decoupling differently: rather than combining a single momentum buffer with the current gradient, it maintains two momentum buffers at distinct decay coefficients and takes a linear combination of the two. The per-step update is given in Algorithm 1.

For the purpose of the correspondence, we analyze the variant with $\beta_1 = 0$, which disables the fast buffer m_1 . Pagliardini et al. (2024) report that this variant nearly matches the full method at small batch sizes, and our experiments in Section 5 confirm this. Under $\beta_1 = 0$, the AdEMAMix update coincides with the accelerated-SGD family of Equation (1), with the slow buffer m_2 playing the role of m_t . We further show that the prescribed schedules for the momentum coefficient β_3 and for α —the coefficient controlling the relative weight on the slow buffer—track the schedules that the theory of accelerated SGD (Gupta et al., 2023) requires.

Specifically, in smooth stochastic convex optimization, acceleration requires a momentum coefficient of the form:

$$\beta_{a,t} = 1 - \frac{k}{t}$$

for some constant $k > 0$, as established by Gupta et al. (2023). AdEMAMix approximates this schedule by annealing β_3 toward 1 over training.

A second point of comparison concerns the normalization of the momentum recursion. Accelerated SGD accumulates gradients in the unnormalized (“theory-style”) form:

$$m_t = \beta_{a,t} m_{t-1} + g_t$$

whereas the slow buffer in Algorithm 1 uses the normalized (EMA-style) recursion:

$$m_2^{(t)} \leftarrow \beta_3^{(t)} m_2^{(t-1)} + (1 - \beta_3^{(t)}) g^{(t)}.$$

These two conventions are reconciled by a change of scale. When $\beta_{a,t}$ scales as $1 - 1/t$, the unnormalized buffer m_t accumulates past gradients with total mass growing linearly in t ; the EMA-style buffer of AdEMAMix compensates through the α coefficient, which is likewise scheduled to grow linearly in t . This alignment between the AdEMAMix schedules and the accelerated-SGD schedules accounts for the empirical advantage of AdEMAMix visible in Figure 1.

The correspondence also predicts the failure mode at large batch sizes. Pagliardini et al. (2024) report that the $\beta_1 = 0.0$ variant degrades in this regime, and the theory of Gupta et al. (2023) explains why: as the gradient noise decreases with batch size, the optimal weight on the current gradient must decrease as well. AdEMAMix instead pins this weight at 1 by construction, so it cannot adapt to the low-noise regime—the likely cause of its large-batch degradation.

Motivated by this analysis, the next section introduces a simplified variant that (i) exposes an explicit, tunable weight on the current gradient, (ii) removes the schedule on α , and (iii) retains a single momentum buffer. We show empirically that this variant matches AdEMAMix in both the small- and large-batch regimes.

Algorithm 1 Single step of AdEMAMix optimizer.

- 1: **Input:** Data distribution $\mathcal{D}$. Initial model parameters $\theta^{(0)}$. Number of iterations T . Learning rate η . ϵ a small constant. AdamW parameters: β_1, β_2 . AdEMAMix parameters β_3, α . Warmup parameter T_{α, β_3} , note that we usually set it to T . β_{start} is usually set to β_1 .
 - 2: Optional: use schedulers $\eta^{(t)}, \beta_3^{(t)} \leftarrow f_{\beta_3}(t, \beta_3, \beta_{\text{start}}, T_{\alpha, \beta_3})$ and $\alpha^{(t)} \leftarrow f_{\alpha}(t, \alpha, T_{\alpha, \beta_3})$
 - 3: Sample batch: $x \sim \mathcal{D}$
 - 4: Compute gradient: $g^{(t)} \leftarrow \nabla_{\theta} \mathcal{L}_{\theta^{(t-1)}}(x)$
 - 5: Update the fast EMA m_1 : $m_1^{(t)} \leftarrow \beta_1 m_1^{(t-1)} + (1 - \beta_1) g^{(t)}$
 - 6: Update the slow EMA m_2 : $m_2^{(t)} \leftarrow \beta_3^{(t)} m_2^{(t-1)} + (1 - \beta_3^{(t)}) g^{(t)}$
 - 7: Update the second moment estimate: $\nu^{(t)} \leftarrow \beta_2 \nu^{(t-1)} + (1 - \beta_2) (g^{(t)})^2$
 - 8: Update parameters: $\theta^{(t)} \leftarrow \theta^{(t-1)} - \eta^{(t)} \left(\frac{\hat{m}_1^{(t)} + \alpha^{(t)} m_2^{(t)}}{\sqrt{\hat{\nu}^{(t)} + \epsilon}} \right)$
-

Algorithm 2 Single step of Simplified AdEMAMix optimizer.

- 1: **Input:** Data distribution $\mathcal{D}$. Initial model parameters $\theta^{(0)}$. Number of iterations T . Learning rate η . ϵ a small constant. AdamW parameters: β_1, β_2 . AdEMAMix parameters $\alpha, \beta_{\text{start}}$. Warmup parameter T_{β_1} , note that we usually set it to T .
 - 2: Optional: use schedulers $\eta^{(t)}, \beta_1^{(t)} \leftarrow f_{\beta_1}(t, \beta_1, \beta_{\text{start}}, T_{\beta_1})$
 - 3: Sample batch: $x \sim \mathcal{D}$
 - 4: Compute gradient: $g^{(t)} \leftarrow \nabla_{\theta} \mathcal{L}_{\theta^{(t-1)}}(x)$
 - 5: Update the EMA m_1 : $m_1^{(t)} \leftarrow \beta_1 m_1^{(t-1)} + g^{(t)}$
 - 6: Update the second moment estimate: $\nu^{(t)} \leftarrow \beta_2 \nu^{(t-1)} + (1 - \beta_2) (g^{(t)})^2$
 - 7: Update parameters: $\theta^{(t)} \leftarrow \theta^{(t-1)} - \eta^{(t)} \left(\frac{m_1^{(t)} + \alpha g^{(t)}}{\sqrt{\hat{\nu}^{(t)} + \epsilon}} \right)$
-

4.5 SIMPLIFIED ADEMAMIX

Building on the preceding analysis, we propose a simplified optimizer that maintains a single momentum buffer and requires no schedule on α . The per-step update is specified in Algorithm 2. Two design choices realize the simplification: the buffer is accumulated in the unnormalized theory-style form $m_1^{(t)} \leftarrow \beta_1 m_1^{(t-1)} + g^{(t)}$ rather than as an EMA, and the current gradient enters the update with a single fixed weight α . The construction subsumes Adam as a special case: setting $\alpha = 0$ recovers standard Adam up to a reparameterization of η and β_1 . Section 5 shows that this single-buffer variant matches AdEMAMix in both the small- and large-batch regimes.

5 EXPERIMENTS

We evaluate on autoregressive language modeling with a 150M-parameter decoder-only transformer trained on the C4 dataset. The training configuration fixes a sequence length of 1024 and a micro-batch of 32 sequences, and processes 15B tokens in total ($\approx 5 \times$ the Chinchilla token budget); the resulting small effective batch places training in the noise-dominated regime in which acceleration is theoretically expected to help.

The comparison spans the following optimizers:

1. Standard AdamW with cosine decay
2. Standard AdamW with weight averaging

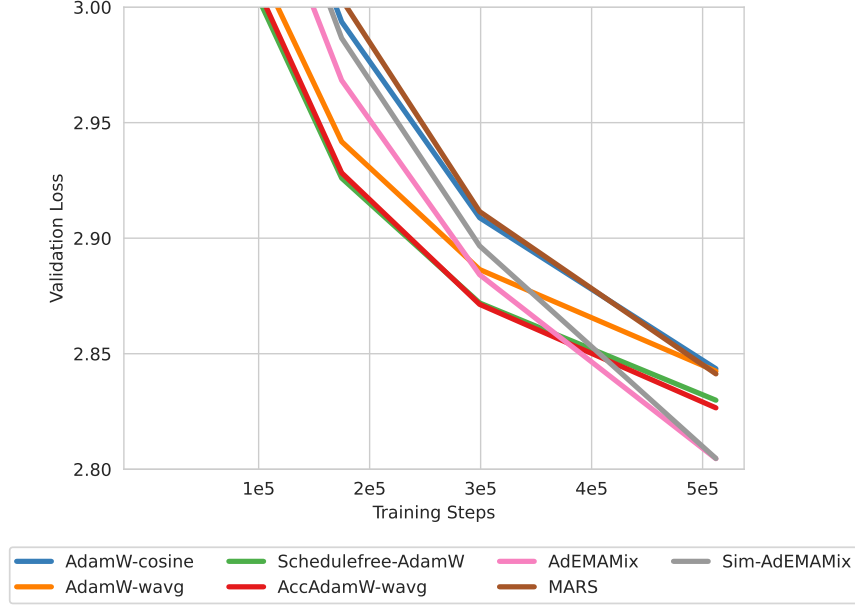


Figure 1: Best-run validation loss for each optimizer of Section 5 on the 150M-parameter decoder-only transformer language-modeling task (small-batch regime). AdEMAMix and Simplified-AdEMAMix attain the lowest loss, consistent with their update rules being the closest instantiation of the accelerated-SGD schedules.

Algorithm 3 Single step of accelerated SGD based Adam with weight averaging. For simplicity we ignore the initialization, other boundary effects such as bias correction, and weight decay. Hyperparameters: Learning rate η , betas = $(\beta_1, \beta_2, \beta_3)$, weight averaging coefficient δ , and epsilon ϵ .

- 1: Sample batch B_t .
- 2: $g \leftarrow -\nabla_w \phi_{B_t}(w_t)$
- 3: $v \leftarrow \beta_2 v + (1 - \beta_2)(g \odot g)$
- 4: $N \leftarrow \frac{\beta_3 m + (1 - \beta_3)g}{\sqrt{v} + \epsilon}$
- 5: $w \leftarrow w - \eta N$
- 6: $m \leftarrow \beta_1 m + (1 - \beta_1)g$
- 7: $c = \max(1 - 1/t, 1 - 1/(\delta t))$
- 8: $w_{\text{avg}} \leftarrow c w_{\text{avg}} + (1 - c)w$

3. Schedule-Free AdamW
4. Accelerated AdamW with weight averaging (Algorithm 3)
5. MARS
6. AdEMAMix
7. Simplified-AdEMAMix

Each optimizer is tuned over the hyperparameter grids detailed in Section A, and we report the best-run validation loss.

The results in Figure 1 confirm the predicted ordering. Schedule-Free AdamW and accelerated AdamW with trailing weight averaging attain nearly identical validation loss, as required by the equivalence derived in the previous section, and both improve on AdamW with cosine decay as well as AdamW with trailing weight averaging. AdEMAMix and Simplified-AdEMAMix attain the lowest loss overall, which we attribute to their update coefficients being the closest instantiation of the accelerated-SGD schedules among the methods compared.

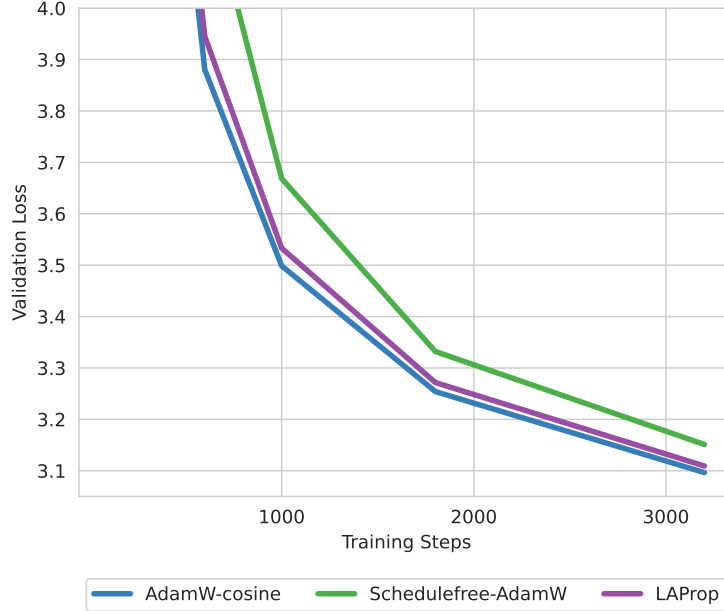


Figure 2: Best-run validation loss for AdamW with cosine decay, Schedule-Free AdamW, and LARProp in the large-batch regime. The near-coincidence of AdamW and LARProp isolates the momentum–preconditioning ordering as a non-factor, leaving the momentum/averaging coupling as the source of the Schedule-Free AdamW gap. Experimental details in Section 5.1.

5.1 LARGE BATCH SIZE EXPERIMENTS

The preceding experiments operated in the small-batch (high-noise) regime. To test whether the accelerated methods retain their advantage once the noise-dominated regime no longer holds, we now increase the effective batch to 1M tokens and train over 3B tokens ($\approx$ the Chinchilla token budget).

Schedule-Free AdamW. Figure 2 shows Schedule-Free AdamW trailing AdamW by a substantial margin. We attribute the gap to the structural coupling between the momentum and weight-averaging coefficients. In the large-batch regime the optimal momentum coefficient is well below $1 - 1/t$; one can restore a smaller coefficient with the parameterization $\approx 1 - r/t$ for $r \geq 1$, but because the same r sets the averaging window, increasing it shortens the effective trailing average—an intrinsic tension the coupling cannot resolve.

A competing explanation is the ordering of momentum and preconditioning: AdamW applies momentum before preconditioning, whereas Schedule-Free AdamW applies preconditioning first, making it algorithmically equivalent to LARProp (Ziyin et al., 2021). We rule this out empirically—Figure 2 shows AdamW and LARProp performing comparably—so the ordering is not the dominant factor, and the momentum/averaging coupling remains the operative cause.

AdEMAMix. Consistent with Pagliardini et al. (2024), Figure 3 shows that the single-buffer configuration $\beta_1 = 0.0$ degrades sharply relative to the two-buffer configuration in the large-batch regime. The mechanism is the one identified in the previous section: AdEMAMix fixes the weight on the current gradient at 1, whereas the accelerated-SGD theory of Gupta et al. (2023) requires this weight to diminish as the batch size grows and the gradient noise shrinks.

By contrast, Figure 3 shows that Simplified-AdEMAMix matches the two-buffer AdEMAMix while maintaining only a single momentum buffer. The matching configuration uses $\alpha = 0.0$, so in this regime Simplified-AdEMAMix reduces exactly to Adam with a momentum schedule—demonstrating that the scheduled momentum, rather than the second buffer or a fixed unit gradient weight, carries the performance.

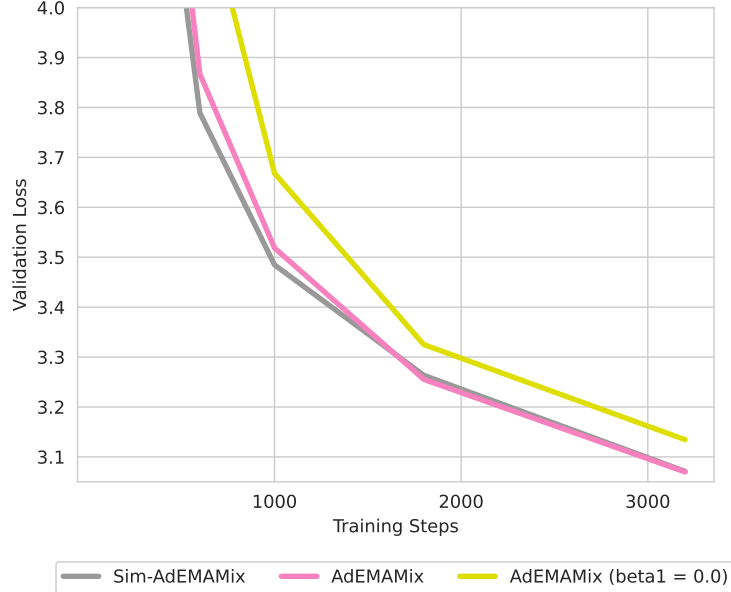


Figure 3: Best-run validation loss for AdEMAMix (with the two-buffer configuration and with the single-buffer $\beta_1 = 0.0$ configuration) and for Simplified-AdEMAMix in the large-batch regime. Experimental details in Section 5.1.

6 CONCLUSION

We established update-rule-level equivalences between the accelerated-SGD family of Equation (1) and several recently proposed optimizers—Schedule-Free methods, AdEMAMix, MARS, and Lion—identifying in each case the coefficient map that realizes the decoupling of the momentum coefficient from the weight on the current gradient. Empirically, AdEMAMix, whose coefficients most closely instantiate the accelerated-SGD schedules, attained the lowest validation loss in the small-batch (noise-dominated) regime, as the correspondence predicts.

Leveraging this correspondence, we introduced Simplified-AdEMAMix, which replaces the two momentum buffers of AdEMAMix with a single unnormalized buffer and an explicit fixed weight on the current gradient. It matches AdEMAMix across both the small- and large-batch regimes while removing the second buffer, and hence the associated optimizer-state overhead.

REFERENCES

- Chen, X., Liang, C., Huang, D., Real, E., Wang, K., Pham, H., Dong, X., Luong, T., Hsieh, C., Lu, Y., and Le, Q. V. Symbolic discovery of optimization algorithms. In Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., and Levine, S. (eds.), *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023. URL http://papers.nips.cc/paper_files/paper/2023/hash/9a39b4925e35cf447ccba8757137d84f-Abstract-Conference.html.
- Defazio, A. Momentum via primal averaging: Theoretical insights and learning rate schedules for non-convex optimization, 2021. URL <https://arxiv.org/abs/2010.00406>.
- Defazio, A., Yang, X., Mehta, H., Mishchenko, K., Khaled, A., and Cutkosky, A. The road less scheduled. *CoRR*, abs/2405.15682, 2024. doi: 10.48550/ARXIV.2405.15682. URL <https://doi.org/10.48550/arXiv.2405.15682>.
- Gupta, K., Siegel, J., and Wojtowysch, S. Achieving acceleration despite very noisy gradients, 2023. URL <https://arxiv.org/abs/2302.05515>.

- Jain, P., Kakade, S. M., Kidambi, R., Netrapalli, P., and Sidford, A. Accelerating stochastic gradient descent for least squares regression. In Bubeck, S., Perchet, V., and Rigollet, P. (eds.), *Proceedings of the 31st Conference On Learning Theory*, volume 75 of *Proceedings of Machine Learning Research*, pp. 545–604. PMLR, 06–09 Jul 2018. URL <https://proceedings.mlr.press/v75/jain18a.html>.
- Kidambi, R., Netrapalli, P., Jain, P., and Kakade, S. M. On the insufficiency of existing momentum schemes for stochastic optimization. In *International Conference on Learning Representations*, 2018. URL <https://openreview.net/forum?id=rJTutzbA->.
- Liu, C. and Belkin, M. Accelerating sgd with momentum for over-parameterized learning. In *International Conference on Learning Representations*, 2020. URL <https://openreview.net/forum?id=rlgixp4FPH>.
- Lucas, J., Sun, S., Zemel, R., and Grosse, R. Aggregated momentum: Stability through passive damping. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=Syxt5oC5YQ>.
- Ma, J. and Yarats, D. Quasi-hyperbolic momentum and adam for deep learning. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=SlfUpoR5FQ>.
- Pagliardini, M., Ablin, P., and Grangier, D. The ademamix optimizer: Better, faster, older. 2024. URL <https://arxiv.org/abs/2409.03137>.
- Porian, T., Wortsman, M., Jitsev, J., Schmidt, L., and Carmon, Y. Resolving discrepancies in compute-optimal scaling of language models. *CoRR*, abs/2406.19146, 2024. doi: 10.48550/ARXIV.2406.19146. URL <https://doi.org/10.48550/arXiv.2406.19146>.
- Vaswani, S., Bach, F., and Schmidt, M. Fast and faster convergence of sgd for over-parameterized models and an accelerated perceptron. In Chaudhuri, K. and Sugiyama, M. (eds.), *Proceedings of the Twenty-Second International Conference on Artificial Intelligence and Statistics*, volume 89 of *Proceedings of Machine Learning Research*, pp. 1195–1204. PMLR, 16–18 Apr 2019. URL <https://proceedings.mlr.press/v89/vaswani19a.html>.
- Yuan, H., Liu, Y., Wu, S., Zhou, X., and Gu, Q. Mars: Unleashing the power of variance reduction for training large models, 2024. URL <https://arxiv.org/abs/2411.10438>.
- Zi Yin, L., Wang, Z. T., and Ueda, M. Laprop: Separating momentum and adaptivity in adam, 2021. URL <https://arxiv.org/abs/2002.04839>.

A HYPERPARAMETERS

We report the swept hyperparameter grids for the small-batch experiments below; for each optimizer we select the configuration minimizing validation loss.

1. AdamW with cosine decay - 51.2k warmup - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.9, 0.95]$, β_2 in $[0.99, 0.999, 0.99968, 0.9999]$. The sweep selected $\beta_1 = 0.9$ and $\beta_2 = 0.999$, coinciding with the library defaults. Whereas $\beta_2 = 0.95$ is standard at larger batch sizes, the advantage of a larger β_2 in the small-batch regime is consistent with the observation of Porian et al. (2024).
2. AdamW with cosine decay - 10k warmup - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, with β_1, β_2 fixed to the sweep optima $\beta_1 = 0.9, \beta_2 = 0.999$ from the preceding configuration. This shorter warmup underperformed the 51.2k-step warmup.
3. AdamW constant fraction weight averaging: - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, $\beta_1 = 0.9, \beta_2$ in $[0.99, 0.997, 0.999, 0.9997]$, δ in $[0.05, 0.1, 0.2]$.
4. AdamW with cosine decay and weight averaging - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, $\beta_1 = 0.9, \beta_2 = 0.999, \delta$ in $[0.025, 0.05, 0.1]$.
5. Accelerated SGD based AdamW with cosine decay - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.999, 0.99968, 0.9999]$, β_2 in $[0.99, 0.9968, 0.999]$, $\beta_3 = 0.9$
6. Accelerated SGD based AdamW with constant learning rate and weight averaging - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.99684, 0.999]$, β_2 in $[0.999]$, $\beta_3 = 0.9, \delta$ in $[0.05, 0.1]$
7. Accelerated SGD based AdamW with cosine decay and weight average - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.99684, 0.999]$, $\beta_2 = 0.999, \delta$ in $[0.05, 0.1], \beta_3 = 0.9$
8. Schedulefree AdamW with constant learning rate - learning rate in $[3.16e-4, 1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.8, 0.9, 0.95]$, $\beta_2 = 0.999$
9. Schedulefree AdamW with cosine decay - $[3.16e-4, 1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.8, 0.9, 0.95]$, $\beta_2 = 0.999$
10. MARS - $[3.16e-4, 1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.9, 0.95, 0.99]$, β_2 in $[0.99, 0.999]$, γ in $[0.0, 0.01, 0.02, 0.03, 0.04, 0.05]$, precondition 1d was set to True.
11. AdEMAMix - $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.0, 0.9]$, $\beta_2 = 0.999, \beta_3$ in $[0.99, 0.999, 0.9999]$, α in $[2, 4, 8, 16]$.
12. Sim-AdEMAMix - $[1e-6, 3.16e-6, 1e-5, 3.16e-5]$, β_1 in $[0.99, 0.999, 0.9999]$, $\beta_2 = 0.999, \alpha$ in $[10, 20, 50, 100]$

The corresponding hyperparameter grids for the large-batch experiments are as follows:

1. Schedule-Free AdamW: $[1e-3, 3.16e-3, 1e-2]$, β in $[0.8, 0.9, 0.95]$, β_2 in $[0.9, 0.95]$, r in $[0.0, 5.0, 9.0, 50.0]$
2. AdamW: $[1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.9, 0.95]$, β_2 in $[0.9, 0.95]$
3. LAMProp: $[1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.9, 0.95]$, β_2 in $[0.9, 0.95]$
4. AdEMAMix: $[1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.0, 0.9]$, $\beta_2 = 0.95, \beta_3$ in $[0.9, 0.95, 0.99]$, α in $[2, 4, 8, 16]$
5. Sim-AdEMAMix: $[1e-4, 3.16e-4, 1e-3]$, β_1 in $[0.9, 0.95, 0.99]$, $\beta_2 = 0.95, \alpha$ in $[0.0, 0.5, 1.0]$

B EQUIVALENCE OF PREVIOUS ACCELERATION METHODS

Recall the general accelerated-SGD form of Equation (1). In this appendix we exhibit, for each of Jain et al. (2018); Vaswani et al. (2019); Liu & Belkin (2020); Gupta et al. (2023), an explicit change of variables that rewrites the published update into this form, thereby establishing membership in the family.

B.1 AGNES

The update of Gupta et al. (2023) is:

$$x'_n = x_n + \alpha v_n \quad x_{n+1} = x'_n - \eta g'_n \quad v_{n+1} = \rho_n(v_n - g'_n)$$

where g'_n is the stochastic gradient evaluated at x'_n and the objective is evaluated at x_n . Eliminating x_n and v_n in favor of x'_n and the rescaled momentum $-v_{n+1}/\rho_n$ yields:

$$x'_{n+1} = x'_n - \eta g'_n + \alpha v_{n+1} \quad -\frac{v_{n+1}}{\rho_n} = \rho_{n-1} \left(-\frac{v_n}{\rho_{n-1}} \right) + g'_n$$

so that x'_{n+1} obeys the momentum-plus-direct-gradient update of Equation (1).

B.2 ASGD

The update of Jain et al. (2018) is:

$$y_{j-1} = \alpha x_{j-1} + (1 - \alpha)v_{j-1} \quad x_j = y_{j-1} - \delta g_{j-1} \quad z_{j-1} = \beta y_{j-1} + (1 - \beta)v_{j-1} \quad v_j = z_{j-1} - \gamma g_{j-1}$$

where g_{j-1} is the stochastic gradient evaluated at y_{j-1} and the objective is evaluated at the tail-averaged x . Eliminating the auxiliary sequences in favor of y_j and the rescaled momentum $(y_{j-1} - v_j)/(\gamma - (1 - \beta)\alpha\delta)$ gives:

$$y_j = y_{j-1} - \alpha\delta g_{j-1} - (1 - \alpha)[y_{j-1} - v_j] \quad \frac{y_{j-1} - v_j}{\gamma - (1 - \beta)\alpha\delta} = (1 - \beta)\alpha \frac{y_{j-2} - v_{j-1}}{\gamma - (1 - \beta)\alpha\delta} + g_{j-1}$$

which is again of the form of Equation (1).

B.3 MASS

The update of Liu & Belkin (2020) is:

$$w_{t+1} = u_t - \eta_1 g_t \quad u_{t+1} = (1 + \gamma)w_{t+1} - \gamma w_t + \eta_2 g_t$$

where g_t is the stochastic gradient evaluated at u_t and the objective is evaluated at w_t . Eliminating w_t in favor of u_t and the rescaled momentum $(w_t - u_t)/(\eta_1\gamma - \eta_2)$ yields:

$$u_{t+1} = u_t - \gamma(w_t - u_t) - [\eta_1(1 + \gamma) - \eta_2]g_t \quad \frac{w_t - u_t}{\eta_1\gamma - \eta_2} = \gamma \frac{w_{t-1} - u_{t-1}}{\eta_1\gamma - \eta_2} + g_t$$

which matches the form of Equation (1).

B.4 SGD WITH NESTEROV ACCELERATION

The update of Vaswani et al. (2019) is:

$$w_{k+1} = \zeta_k - \eta g_k \quad \zeta_k = \alpha_k v_k + (1 - \alpha_k)w_k \quad v_{k+1} = \beta_k v_k + (1 - \beta_k)\zeta_k - \gamma_k \eta g_k$$

Eliminating w_k and v_k in favor of ζ_k and the momentum gap $v_{k+1} - w_{k+1}$ gives:

$$\zeta_{k+1} = \zeta_k - \eta g_k + \alpha_{k+1}[v_{k+1} - w_{k+1}]; \quad v_{k+1} - w_{k+1} = \beta_k(1 - \alpha_k)[v_k - w_k] - \eta(\gamma_k - 1)g_k$$

which is once more an instance of Equation (1).